

06

CAPÍTULO

Arquitetura de Sistemas Inteligentes

Como transformar placas embarcadas em soluções profissionais: Edge Computing, Fog, Cloud, protocolos, escalabilidade, segurança e dez estudos arquiteturais completos.

Este capítulo ensina a projetar arquiteturas completas, utilizando as plataformas UNIIKER como elementos de um ecossistema maior. Da pirâmide da computação (sensor → decisão) a dez estudos arquiteturais (rios amazônicos, hospitais, cidades inteligentes), o leitor aprende a integrar K10, M10 e Cloud em soluções escaláveis, seguras e interoperáveis — capazes de evoluir de uma placa a redes nacionais.

ARQUITETURA

EDGE COMPUTING

IOT ESCALÁVEL

FOG · CLOUD

PROTOCOLOS

SEGURANÇA

Sumário do Capítulo

6.1 O que é Arquitetura de Sistemas Inteligentes?	3
6.1.1 Conceitos Fundamentais	3
6.1.2 AI Pipeline, Data Pipeline e Digital Twin	4
6.1.3 Por que pensar em arquitetura antes do código?	4
6.2 A Pirâmide da Computação	4
6.2.1 Camadas da Pirâmide	5
6.3 Onde K10 e M10 se encaixam?	6
6.3.1 K10 como Edge Node	6
6.3.2 M10 como Edge Gateway / Fog Node	7
6.4 Arquiteturas de Referência	7
6.4.1 IoT Industrial	7
6.4.2 Agricultura Inteligente	8
6.4.3 Smart City	9
6.4.4 Robótica	10
6.4.5 Saúde	11
6.4.6 Indústria 4.0	12
6.4.7 Monitoramento Ambiental	13
6.5 Protocolos de Comunicação	14
6.5.1 Protocolos de Aplicação	15
6.5.2 Protocolos de Enlace e Físicos	15
6.5.3 Protocolos de Barramento Local	15
6.6 Fluxos de Dados	16
6.6.1 Anatomia do Fluxo	17
6.7 Arquiteturas de IA	17
6.7.1 Camadas de IA	18
6.8 Integração com Plataformas	19
6.8.1 Plataformas Open Source Self-Hosted	19
6.8.2 Plataformas Cloud	20

6.9 Escalabilidade	20
6.9.1 Gargalos em Cada Nível	21
6.10 Segurança	21
6.10.1 Camadas de Segurança	22
6.11 Engenharia das Decisões Arquiteturais	23
6.11.1 Centralizar ou Distribuir?	23
6.11.2 Cloud, Edge, TinyML?	23
6.12 Estudos Arquiteturais	23
6.12.1 Monitoramento de Rios Amazônicos	23
6.12.2 Monitoramento de ETA	24
6.12.3 Monitoramento de Florestas	24
6.12.4 Smart Campus Universitário	25
6.12.5 Fábrica Inteligente	25
6.12.6 Hospital Inteligente	25
6.12.7 Cidade Inteligente	26
6.12.8 Agricultura de Precisão	26
6.12.9 Robótica Educacional	27
6.12.10 Assistente IA Offline	27
6.13 Conclusão	27
6.13.1 Protótipo vs Sistema de Engenharia	27
6.13.2 Por que arquitetura é mais importante que código?	28
Referências Bibliográficas	28

CAPÍTULO 6

Arquitetura de Sistemas Inteligentes

Como transformar placas embarcadas em soluções profissionais

Após compreender profundamente o hardware do K10 (Capítulo 3) e do M10 (Capítulo 4), e dominar o framework de escolha entre eles (Capítulo 5), este capítulo eleva o nível de abstração: trata as plataformas UNIIKER não como produtos isolados, mas como elementos de arquiteturas maiores de sistemas inteligentes. A pirâmide da computação (do sensor à tomada de decisão), dez arquiteturas de referência, protocolos de comunicação, escalabilidade de uma placa a redes nacionais, segurança em camadas e dez estudos arquiteturais completos preparam o leitor para projetar soluções robustas de IoT e Edge AI em escala profissional.

6.1 O que é Arquitetura de Sistemas Inteligentes?

Arquitetura de Sistemas Inteligentes é a disciplina que estuda como organizar componentes de hardware, software, comunicação e dados em sistemas que percebem o ambiente, processam informação, tomam decisões (frequentemente com IA) e atuam sobre o mundo físico — tudo de forma coordenada, escalável, segura e manutenível. Não se trata de uma única tecnologia, mas da síntese de múltiplas: sistemas embarcados, sistemas distribuídos, Edge e Cloud Computing, pipelines de IA, digital twins e mais. Neste capítulo, o engenheiro aprende a articular esses elementos em soluções coerentes, com as plataformas UNIIKER K10 e M10 assumindo papéis específicos em arquiteturas maiores.

6.1.1 Conceitos Fundamentais

Cinco conceitos estruturam a arquitetura de sistemas inteligentes. Primeiro, **sistema embarcado**: sistema computacional dedicado a uma função específica, com recursos limitados, frequentemente operando em tempo real. O K10 é um sistema embarcado clássico. Segundo, **sistema distribuído**: conjunto de computadores autônomos que aparecem ao usuário como um único sistema coerente, comunicando-se por rede. Arquiteturas IoT com múltiplos K10, M10 e Cloud formam sistemas distribuídos. Terceiro, **Edge Computing**: paradigma que desloca processamento do centro (cloud) para a periferia (dispositivos finais), reduzindo latência, custo de transmissão e dependência de conectividade.

Quarto, **Fog Computing**: camada intermediária entre Edge e Cloud, tipicamente implementada por gateways locais (M10) que agregam dados de múltiplos dispositivos Edge, executam processamento intermediário e forward seletivo para Cloud. Quinto, **Cloud Computing**: infraestrutura escalável de processamento e armazenamento, oferecida como serviço por provedores como AWS, Azure, Google Cloud. Em sistemas inteligentes modernos, esses três níveis (Edge, Fog, Cloud) coexistem e cooperam: dados críticos são processados em Edge (latência baixa), agregados em Fog (redução de volume), armazenados e analisados em Cloud (escala e persistência).

6.1.2 AI Pipeline, Data Pipeline e Digital Twin

Sobre essa infraestrutura distribuída operam dois fluxos fundamentais. **Data Pipeline**: sequência de etapas que transforma dados brutos de sensores em informação útil — aquisição, pré-processamento, agregação, storage, analytics, visualização. **AI Pipeline**: variantes específicas para modelos de IA — coleta de dados de treinamento, pré-processamento, rotulagem, treinamento (em Cloud), validação, quantização, implantação em Edge (K10/M10), monitoramento de drift. **Digital Twin**, conceito fundamental da Indústria 4.0, é a representação digital em tempo real de um ativo físico — um motor industrial, uma cidade, um paciente — sincronizada continuamente com dados de sensores. O digital twin permite simulação, predição e otimização sem interferir no ativo físico.

■ ENGENHARIA EM FOCO

Arquitetura em camadas é o princípio organizador de sistemas inteligentes. Cada camada (física, edge, fog, cloud, aplicação) abstrai a complexidade da camada inferior e oferece serviços à camada superior. Esse particionamento permite: (i) escalar horizontalmente cada camada independentemente; (ii) substituir tecnologias em uma camada sem afetar as demais; (iii) aplicar segurança em múltiplos níveis; (iv) reduzir complexidade cognitiva do projeto. Sem camadas, sistemas distribuídos com milhares de dispositivos tornam-se inadministráveis.

6.1.3 Por que pensar em arquitetura antes do código?

A pergunta ‘por que arquitetura antes de código?’ tem resposta objetiva: decisões arquiteturais tomadas no início do projeto são ordens de magnitude mais baratas de reverter que decisões tomadas em código já escrito. Mudar a escolha entre MQTT e HTTP antes de programar custa uma reunião. Mudar após mil linhas de código custa semanas de refatoração. Mudar após deploy em produção pode custar milhões (recalls, indisponibilidade, perda de confiança). Arquitetura é essencialmente gestão de risco: antecipa decisões que serão caras de mudar e as toma explicitamente, com justificativa documentada. Projetos que pulam a fase arquitetural — ‘vamos programar e ver no que dá’ — tipicamente encontram problemas fundamentais tarde demais: descobrem que o protocolo escolhido não escala, que a plataforma não tem RAM suficiente, que a segurança é inadequada, que a latência é inaceitável.

◆ INSIGHT DO ENGENHEIRO

A regra de Barry Boehm, um dos fundadores da engenharia de software empírica, é clara: o custo de corrigir um erro arquitetural cresce exponencialmente com a fase do projeto em que é descoberto. Erro detectado na fase de requisitos custa 1x; na fase de design, 5x; em codificação, 10x; em testes, 50x; em produção, 200x. Projetos que pulam arquitetura ‘para economizar tempo’ tipicamente gastam 5-10x mais tempo total — e ainda entregam sistemas frágeis.

6.2 A Pirâmide da Computação

A pirâmide da computação é o modelo organizador fundamental de sistemas inteligentes. Visualiza a hierarquia de processamento desde sensores físicos (base, larga, baixa latência, alto volume) até tomada de decisão estratégica (topo, estreita, alta latência, baixo volume). Cada nível tem papel específico, tecnologias

próprias e trade-offs distintos. Compreender essa pirâmide é pré-requisito para qualquer projeto arquitetural sério.

Pirâmide da Computação: do Sensor à Tomada de Decisão

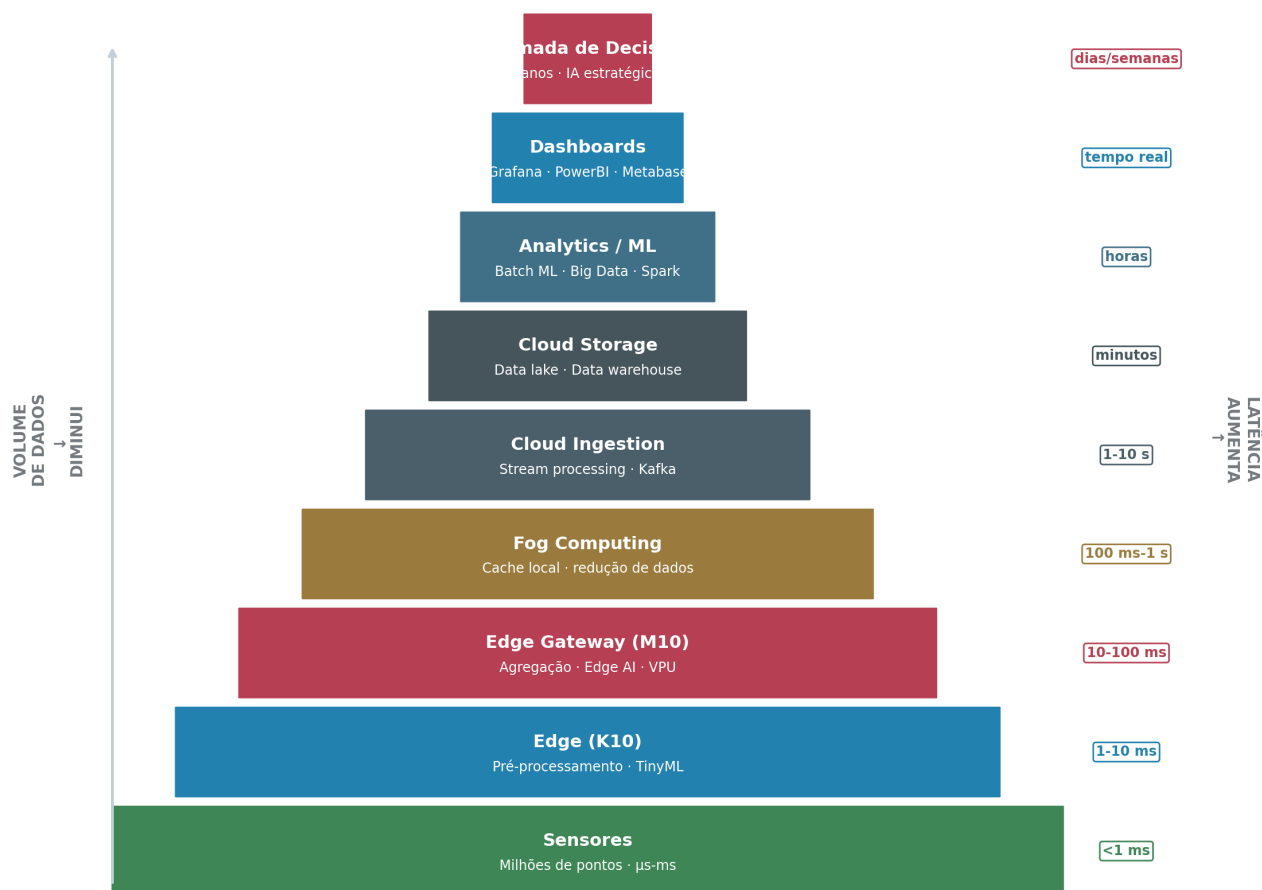


Figura 6.1 — Pirâmide da computação: do sensor à tomada de decisão, em nove camadas. Volume de dados diminui com a altura; latência aumenta. Plataformas UNIHAKER ocupam as camadas Edge (K10) e Fog/Gateway (M10). Fonte: elaborado pelo autor.

6.2.1 Camadas da Pirâmide

A **camada de Sensores** (base da pirâmide) compreende milhões a bilhões de pontos de captura física: temperatura, umidade, pressão, luz, som, movimento, gases, posicionamento. Latência sub-milissegundo, volume de dados bruto imenso. Tecnologias: termistores, strain gauges, MEMS (MPU6050), fotodiodos, microfones, câmeras. A **camada Edge**, onde se posiciona o K10, executa pré-processamento e TinyML: filtra ruído, detecta eventos, comprime dados, toma decisões locais. Latência 1-10 ms, volume reduzido em 10-100x. A **camada Gateway/Fog**, onde se posiciona o M10, agrega dados de múltiplos dispositivos Edge, executa Edge AI mais complexa, mantém histórico local, oferece dashboards. Latência 10-100 ms.

A **camada Cloud Ingestion** recebe streams de dados via Kafka, Kinesis, Pub/Sub. Latência 1-10 s. A **camada Cloud Storage** persiste dados em data lakes (S3, GCS, ADLS) e data warehouses (BigQuery, Snowflake). Latência de minutos a horas. A **camada Analytics/ML** executa treinamento de modelos em batch

com Spark, Databricks, SageMaker. Latência de horas. A **camada de Dashboards** apresenta dados em tempo real via Grafana, PowerBI, Metabase. Por fim, a **camada de Tomada de Decisão**, frequentemente humana, consome dashboards e relatórios para decisões estratégicas em horizontes de dias a semanas.

 **BENCHMARK**

Comparação de volume e latência por camada (sistema típico com 10.000 sensores): Sensores geram 10 GB/dia · Edge (K10) filtra para 100 MB/dia · Fog (M10) agrega para 10 MB/dia · Cloud recebe 10 MB/dia · Analytics processa em 1 hora · Dashboard atualiza em 5 segundos · Decisão humana leva dias. Sem Edge/Fog, Cloud precisaria processar 10 GB/dia — 1000x mais caro em storage e processamento.

6.3 Onde K10 e M10 se encaixam?

K10 e M10 ocupam posições complementares na pirâmide da computação. Esta seção detalha onde cada um se encaixa, quando usar cada um e quando usar ambos em conjunto.

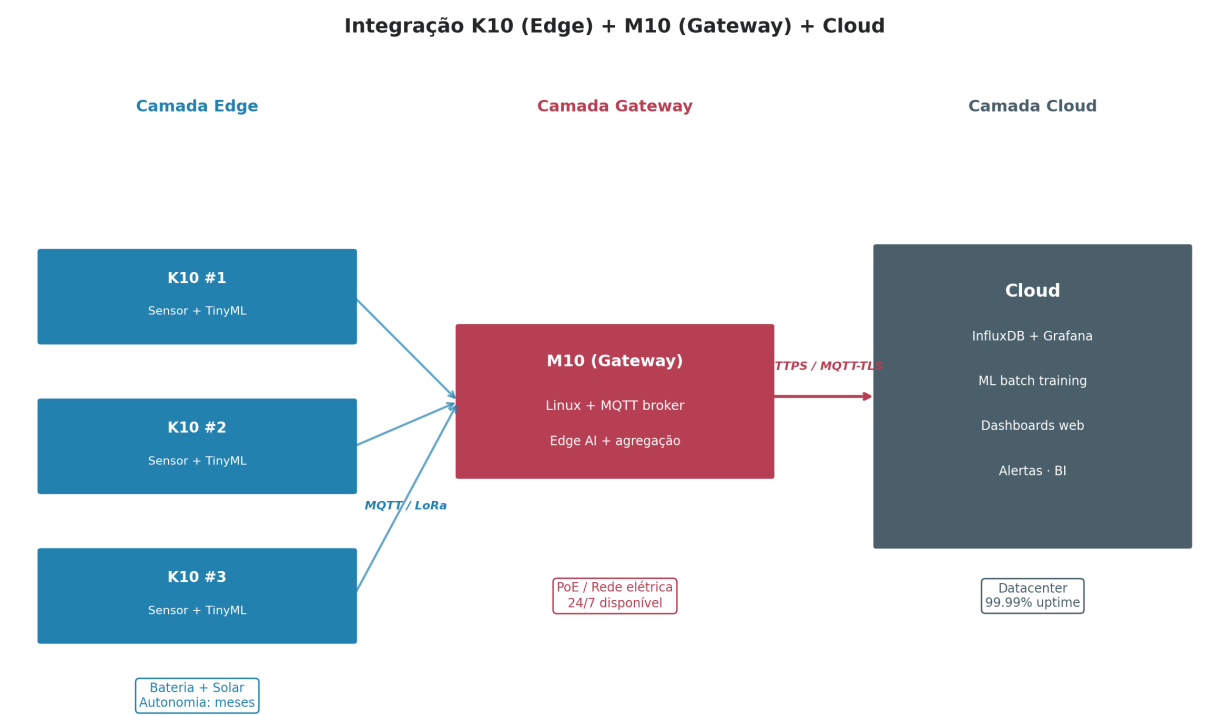


Figura 6.2 — Integração de K10 (camada Edge, baixo consumo) com M10 (camada Gateway, Linux completo) e Cloud. K10s operam em modo bateria/solar, transmitem via MQTT/LoRaWAN para M10 que agrega e forward para Cloud. Fonte: elaborado pelo autor.

6.3.1 K10 como Edge Node

O K10, por seu baixo consumo (10 μ A em Deep Sleep), autonomia solar de meses e boot instantâneo, é a escolha natural para a camada Edge em implantações distribuídas. Cada K10 atua como nó sensorial inteligente: captura dados físicos, executa TinyML para detectar eventos relevantes, transmite via MQTT

(Wi-Fi) ou LoRaWAN (rádio de longo alcance) apenas os dados significativos — não o stream completo. Essa estratégia de ‘filtrar localmente, transmitir seletivamente’ reduz drasticamente o custo de comunicação e energia, viabilizando redes com milhares de nós alimentados por bateria.

6.3.2 M10 como Edge Gateway / Fog Node

O M10, por seu Linux completo, 1 GB de RAM, VPU H.264 e múltiplas interfaces (Ethernet, Wi-Fi, BLE), é a escolha natural para a camada Gateway/Fog. Cada M10 agrega dados de 10-100 K10s vizinhos, executa Edge AI mais complexa (OpenCV, MediaPipe, YOLO), mantém histórico local em InfluxDB, oferece dashboards Grafana para usuários locais, e forward seletivamente para Cloud os dados agregados. Essa arquitetura em camadas é fundamental para escalabilidade: em vez de 10.000 K10s transmitindo diretamente para Cloud (sobrecarga de rede e custo proibitivo), 100 M10s agregam dados de 100 K10s cada, e apenas 100 streams chegam à Cloud.

TRADE-OFF DE ENGENHARIA

Quando usar apenas K10 (sem M10)? Projetos pequenos (1-10 dispositivos) onde escala não justifica gateway dedicado. K10 pode publicar diretamente em Cloud via Wi-Fi. **Vantagem:** custo mínimo, simplicidade. **Desvantagem:** sem agregação local, sem dashboard offline, Cloud congestionada se escala crescer.

Quando usar apenas M10 (sem K10)? Projetos com poucos sensores conectados diretamente ao M10 via GPIO/I²C. Útil para estações fixas com rede elétrica. **Vantagem:** Linux completo, OpenCV, dashboards locais. **Desvantagem:** consumo 2W impede uso a bateria, custo maior por nó.

Quando usar ambos? Projetos médios e grandes (>10 dispositivos) onde K10 edge + M10 gateway + Cloud é arquitetura ótima. **Vantagem:** escala, autonomia, resiliência. **Desvantagem:** complexidade adicional, requer expertise em ambos.

6.4 Arquiteturas de Referência

Esta seção apresenta sete arquiteturas de referência para domínios distintos. Cada arquitetura é genérica o suficiente para adaptar a casos específicos, mas concreta o suficiente para servir como ponto de partida. Estudos de caso mais detalhados aparecem na Seção 6.12.

6.4.1 IoT Industrial

IoT Industrial (IIoT) monitora e otimiza processos manufatureiros. Sensores de vibração, temperatura, corrente elétrica e pressão são lidos por K10s em cada máquina, executando TinyML para detectar anomalias. M10s agregam dados de múltiplas máquinas em uma linha de produção, executam Node-RED para integração com SCADA via OPC-UA, mantém InfluxDB local. Cloud armazena histórico, executa analytics para manutenção preditiva e integra com ERP/MES.

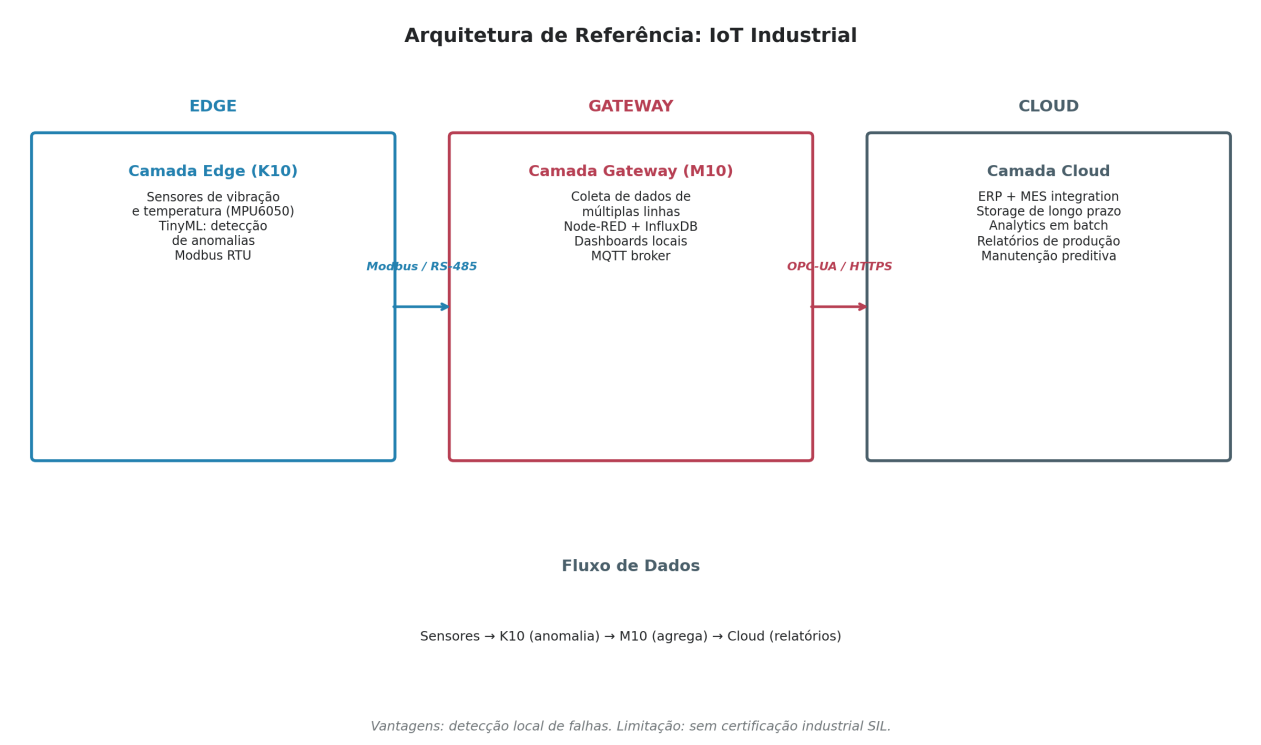


Figura 6.3 — Arquitetura de referência para IoT Industrial. K10 monitora máquinas (TinyML), M10 agrega por linha de produção (Node-RED, OPC-UA), Cloud integra com ERP/MES. Fonte: elaborado pelo autor.

6.4.2 Agricultura Inteligente

Agricultura inteligente (smart agri) usa sensores distribuídos em lavouras para otimizar irrigação, detectar pragas e prever colheita. K10s solares medem umidade do solo, temperatura, umidade do ar, radiação solar. M10s regionais agregam dados via LoRaWAN, executam algoritmos de previsão de irrigação, controlam válvulas solenoides via GPIO. Cloud integra dados de satélite (NDVI) e previsão meteorológica para analytics de safra.

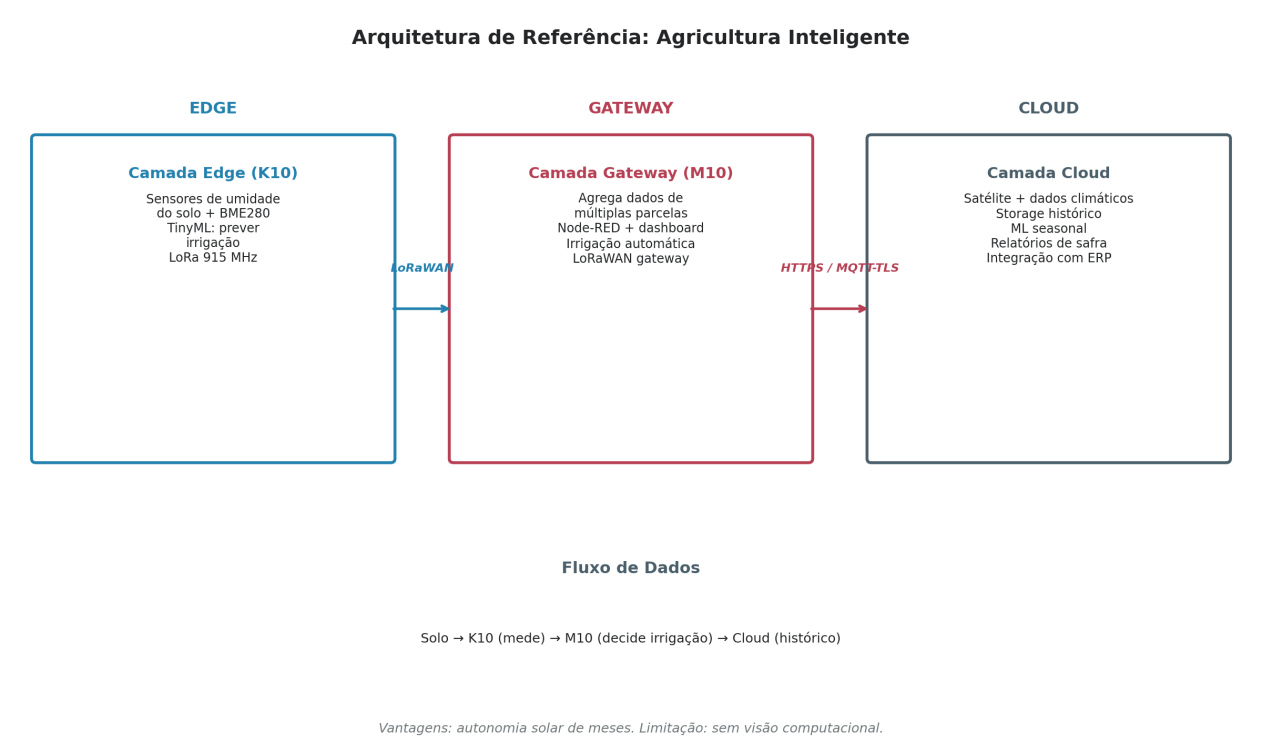


Figura 6.4 — Arquitetura de referência para Agricultura Inteligente. K10 solar mede umidade do solo, M10 regional decide irrigação via LoRaWAN, Cloud integra satélite e clima. Fonte: elaborado pelo autor.

6.4.3 Smart City

Smart City monitora e otimiza serviços urbanos: qualidade do ar, trânsito, ruído, iluminação, resíduos. K10s em postes e cruzamentos medem PM2.5, CO2, ruído, contagem de veículos. M10s por bairro agregam via LoRaWAN ou NB-IoT. Cloud centraliza dados metropolitanos, oferece dashboards públicos e APIs para pesquisa.

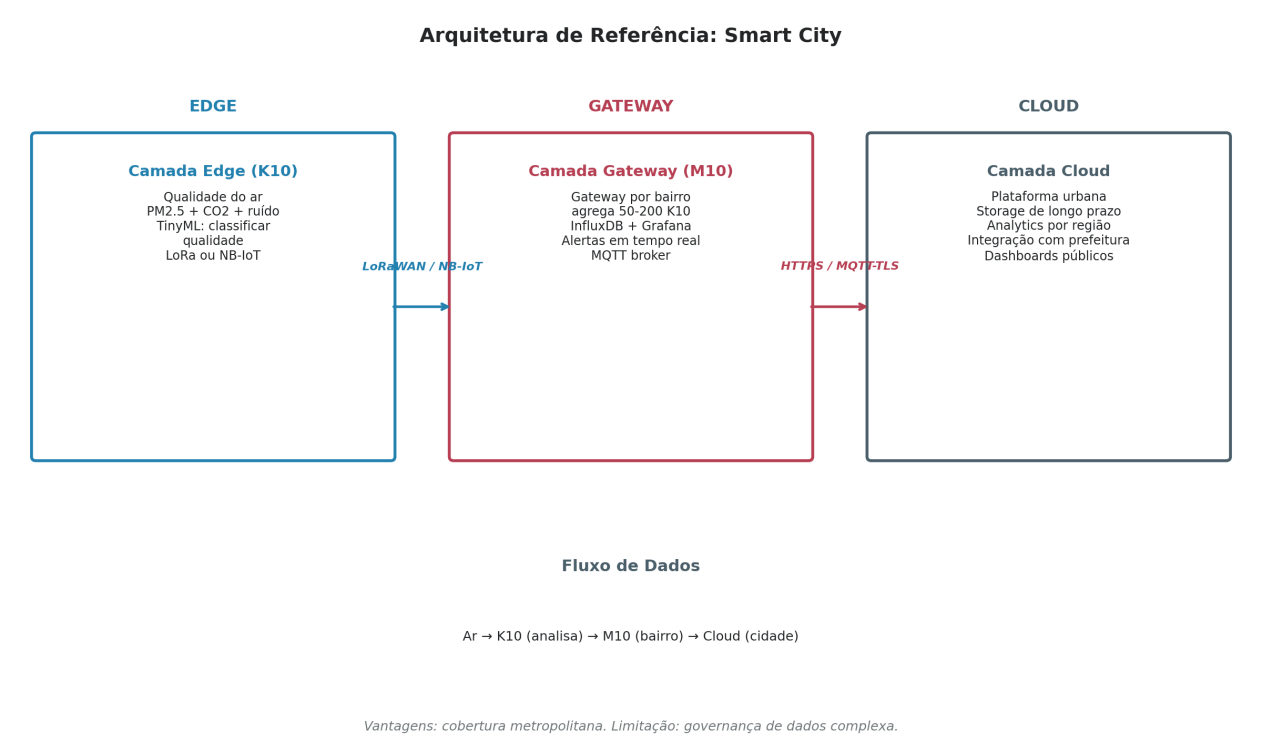


Figura 6.5 — Arquitetura de referência para Smart City. K10 em postes mede ar/ruído, M10 por bairro agrega, Cloud metropolitano integra com prefeitura. Fonte: elaborado pelo autor.

6.4.4 Robótica

Robótica moderna combina percepção (visão, LIDAR), decisão (IA, planejamento) e atuação (motores). K10s podem atuar como controladores de baixo nível (leitura de encoders, controle PID de motores). M10s como cérebros de alto nível (visão computacional, planejamento de trajetória, coordenação multi-robô). Cloud para simulação, treinamento de políticas RL e fleet management.

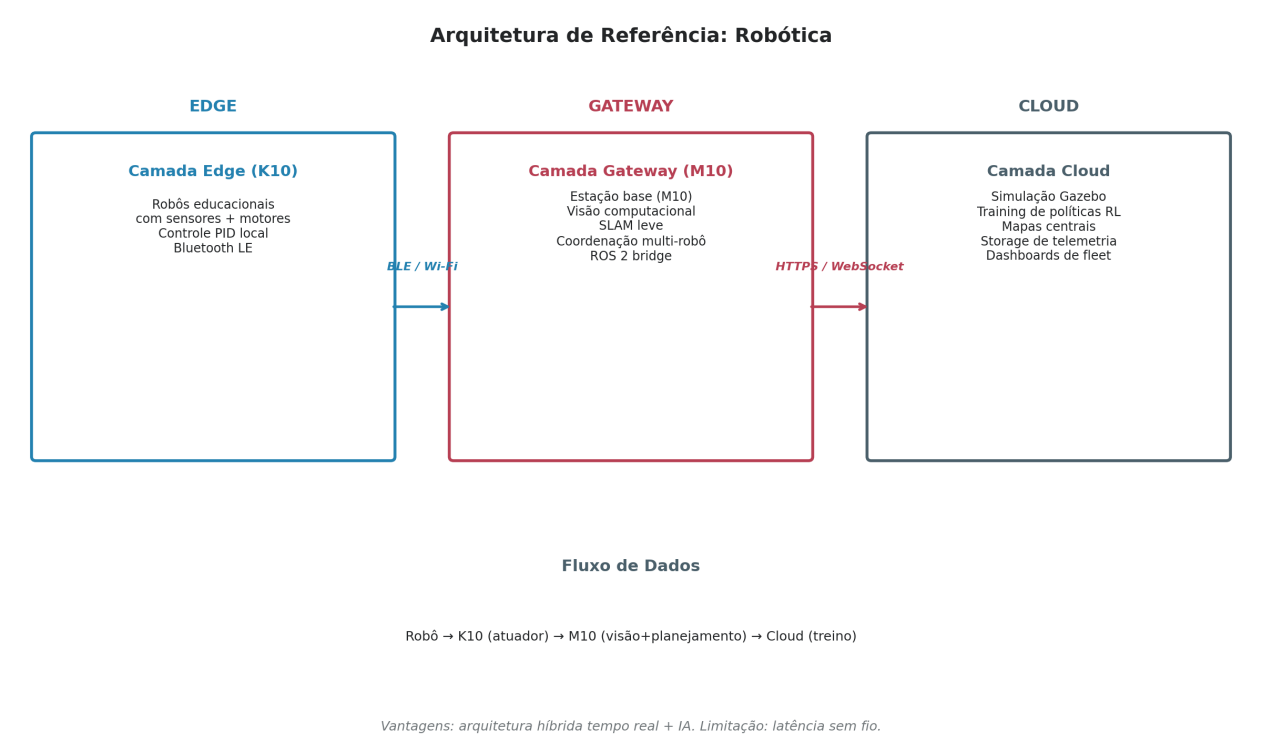


Figura 6.6 — Arquitetura de referência para Robótica. K10 controla atuadores em tempo real, M10 executa visão e planejamento, Cloud treina políticas. Fonte: elaborado pelo autor.

6.4.5 Saúde

Saúde digital usa wearables e dispositivos domiciliares para monitoramento contínuo de pacientes. K10s em wearables medem PPG (frequência cardíaca), acelerômetro (atividade), SpO2, executam TinyML para detectar arritmias. M10s como hubs residenciais agregam dados de múltiplos wearables, executam alertas locais. Cloud hospitalar armazena histórico com criptografia LGPD/HIPAA, oferece dashboards para profissionais.

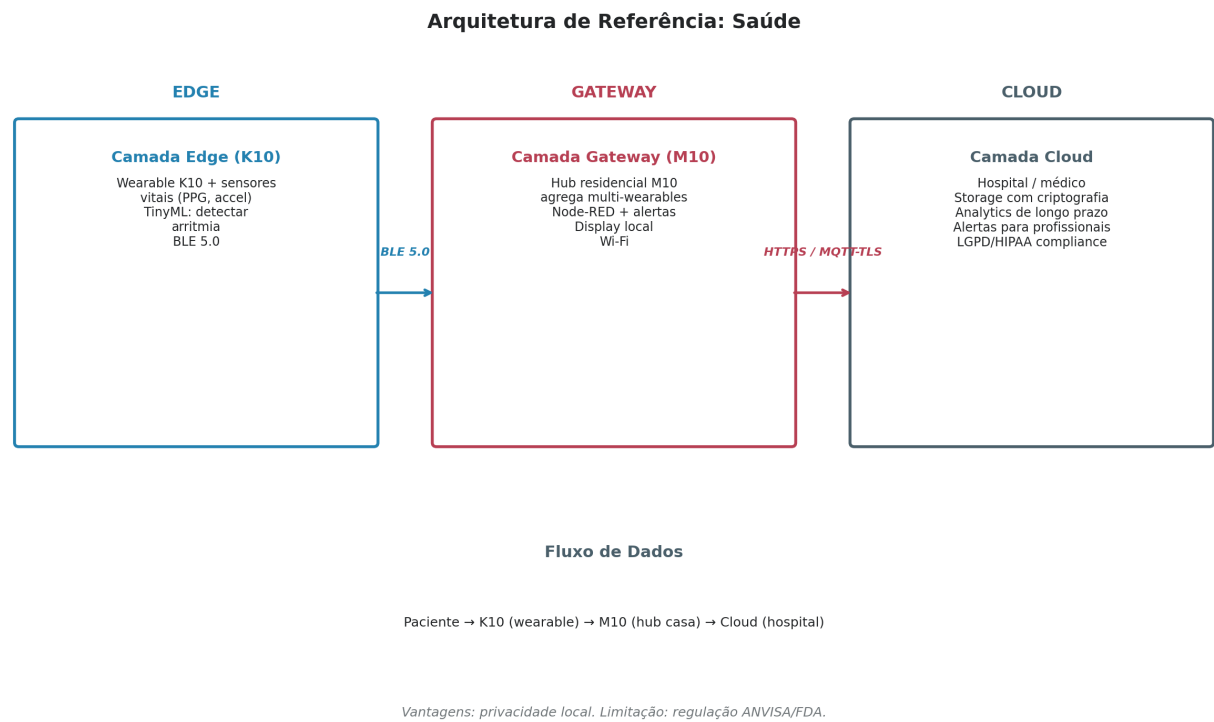


Figura 6.7 — Arquitetura de referência para Saúde Digital. K10 wearable mede sinais vitais, M10 hub residencial agrega, Cloud hospitalar armazena com criptografia. Fonte: elaborado pelo autor.

6.4.6 Indústria 4.0

Indústria 4.0 é a convergência de OT (Operational Technology) e IT (Information Technology) em fábricas inteligentes. K10s com Modbus RTU leem sensores industriais em máquinas. M10s com Docker executam containers de tempo real e IA, fazem bridge OT-IT via MQTT Sparkplug B. Cloud mantém digital twin em tempo real da fábrica inteira, integra com MES (Manufacturing Execution System) e ERP.

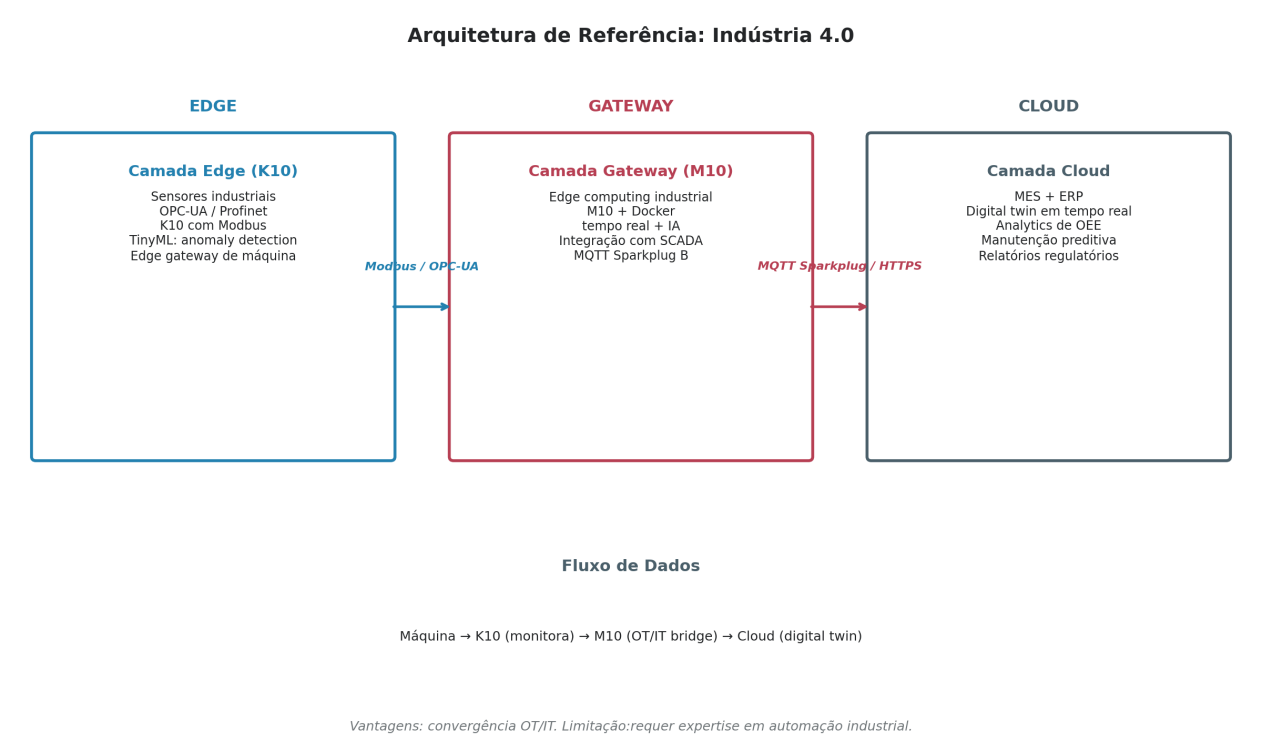


Figura 6.8 — Arquitetura de referência para Indústria 4.0. K10 faz Modbus com máquinas, M10 (Docker) faz bridge OT-IT, Cloud mantém digital twin integrado com MES/ERP. Fonte: elaborado pelo autor.

6.4.7 Monitoramento Ambiental

Monitoramento ambiental em larga escala (florestas, rios, áreas remotas) exige autonomia energética e comunicação de longo alcance. K10s solares medem parâmetros ambientais, transmitem via LoRaWAN Classe A (baixíssimo consumo). M10s em torres regionais fazem concentrator LoRaWAN, agregam 100+ K10s. Cloud nacional mantém histórico multi-anual, modelos preditivos climáticos.

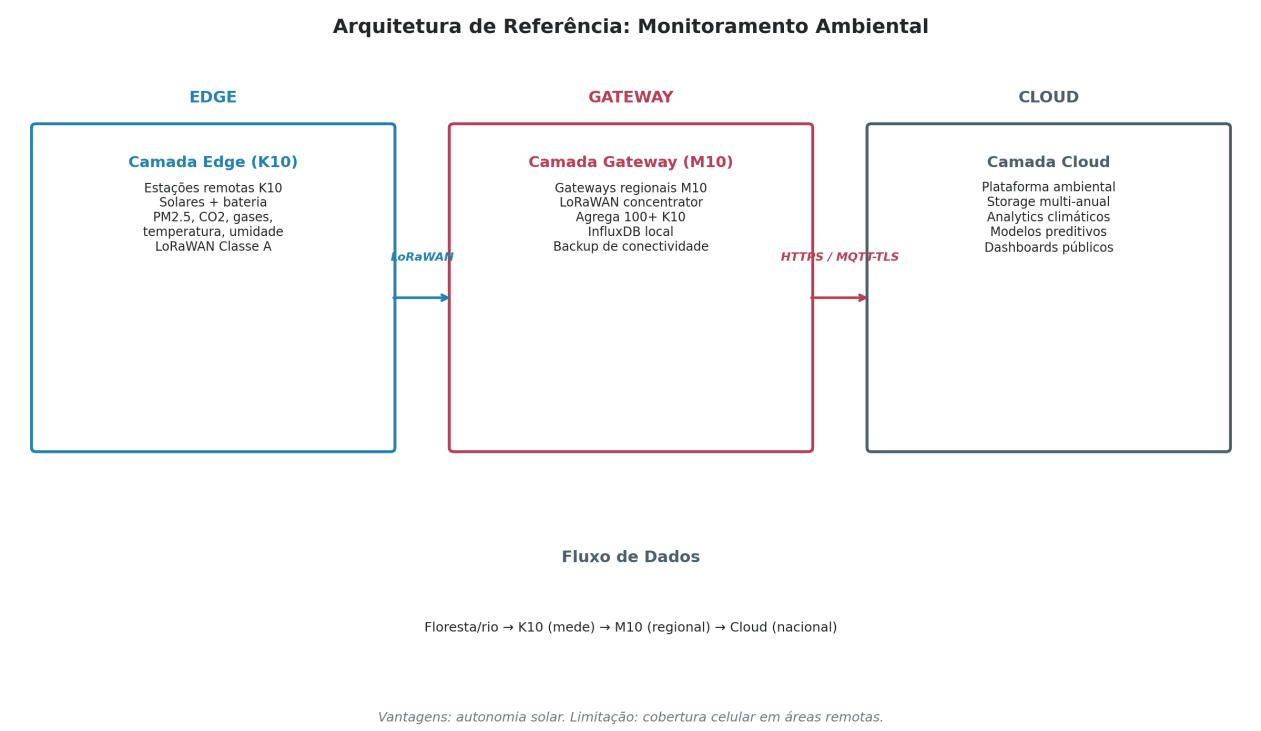


Figura 6.9 — Arquitetura de referência para Monitoramento Ambiental. K10 solar em florestas/rios, M10 regional em torres, Cloud nacional com modelos climáticos. Fonte: elaborado pelo autor.

6.5 Protocolos de Comunicação

Protocolos de comunicação são a cola que conecta componentes em sistemas distribuídos. Esta seção analisa os principais protocolos relevantes para sistemas UNIHAKER, organizados por camada OSI.



Figura 6.10 — Pilha de protocolos por camada OSI (adaptada), mostrando onde cada protocolo relevante para sistemas UNIHAKER se posiciona. Fonte: elaborado pelo autor.

6.5.1 Protocolos de Aplicação

MQTT (Message Queuing Telemetry Transport) é o protocolo de facto para IoT. Lightweight (cabeçalho de 2 bytes), pub/sub com tópicos hierárquicos, QoS 0/1/2, overhead mínimo. K10 e M10 suportam via paho-mqtt e mosquitto broker. Indicado para telemetria esporádica de sensores. **HTTP/REST** é onipresente, simples, stateless, mas mais pesado que MQTT. Útil para integrações com APIs web (RESTful). **CoAP** (Constrained Application Protocol) é a versão REST para dispositivos restritos, sobre UDP, ideal para K10s. **WebSocket** permite comunicação bidirecional full-duplex sobre TCP, útil para dashboards em tempo real. **OPC-UA** é padrão industrial para integração OT-IT, com modelo de informação rico.

6.5.2 Protocolos de Enlace e Físicos

Wi-Fi (802.11 b/g/n) é onipresente em ambientes com infraestrutura, alcance ~30 m interno, throughput até 150 Mbps. Ambos K10 e M10 suportam. **BLE** (Bluetooth Low Energy 5.0) é otimizado para baixo consumo, alcance ~30 m, throughput 2 Mbps, ideal para wearables. **LoRa** e **LoRaWAN** oferecem alcance de quilômetros com consumo ultra-baixo, throughput de 0,3-50 kbps, ideal para sensores remotos. **Ethernet** 100M (apenas M10) oferece comunicação estável e segura para ambientes industriais. **CAN** é padrão automotivo/industrial para comunicação entre microcontroladores. **Modbus RTU** é protocolo serial clássico para PLCs e instrumentação industrial.

6.5.3 Protocolos de Barramento Local

I²C (Inter-Integrated Circuit) é barramento serial de 2 fios, 100 kHz-1 MHz, para sensores próximos ao microcontrolador. **SPI** (Serial Peripheral Interface) é mais rápido (até 50 MHz), 4 fios, para displays, SD, memórias. **UART** é serial assíncrono clássico, para comunicação ponto-a-ponto com módulos GPS, radios, microcontroladores externos. **USB** (apenas M10 via USB-C OTG) permite conectar câmeras UVC, teclados, pen drives.

Protocolo	Camada OSI	Aplicação Típica	K10	M10
MQTT	Aplicação	Telemetria IoT pub/sub	Sim	Sim
HTTP/REST	Aplicação	Integração com APIs web	Sim	Sim
CoAP	Aplicação	IoT em dispositivos restritos	Sim	Sim
WebSocket	Aplicação	Dashboards em tempo real	Sim	Sim
OPC-UA	Aplicação	Integração industrial OT-IT	Não	Sim
Wi-Fi	Enlace	Conectividade local	Sim	Sim
BLE 5.0	Enlace	Wearables, pairing	Sim	Sim
LoRaWAN	Enlace	Sensores remotos	Sim	Sim
Ethernet	Enlace/Fís	Indústria, estabilidade	Não	Sim
CAN	Enlace/Fís	Automotivo, robótica	Não	Não (USB)
Modbus RTU	Aplicação	PLCs, instrumentação	Sim	Sim
I ² C	Enlace	Sensores próximos	Sim	Sim
SPI	Enlace	Displays, SD, memórias	Sim	Sim
UART	Enlace	GPS, rádios, MCUs externos	Sim	Sim
USB 2.0	Física	Câmeras, teclados, pen drive	Não	Sim

Tabela 6.1 — Protocolos relevantes e suporte em K10 e M10. Fonte: elaborado pelo autor.

6.6 Fluxos de Dados

Esta seção apresenta o fluxo completo de dados em sistemas inteligentes baseados em UNIHAKER, desde o sensor físico até a aplicação final do usuário.

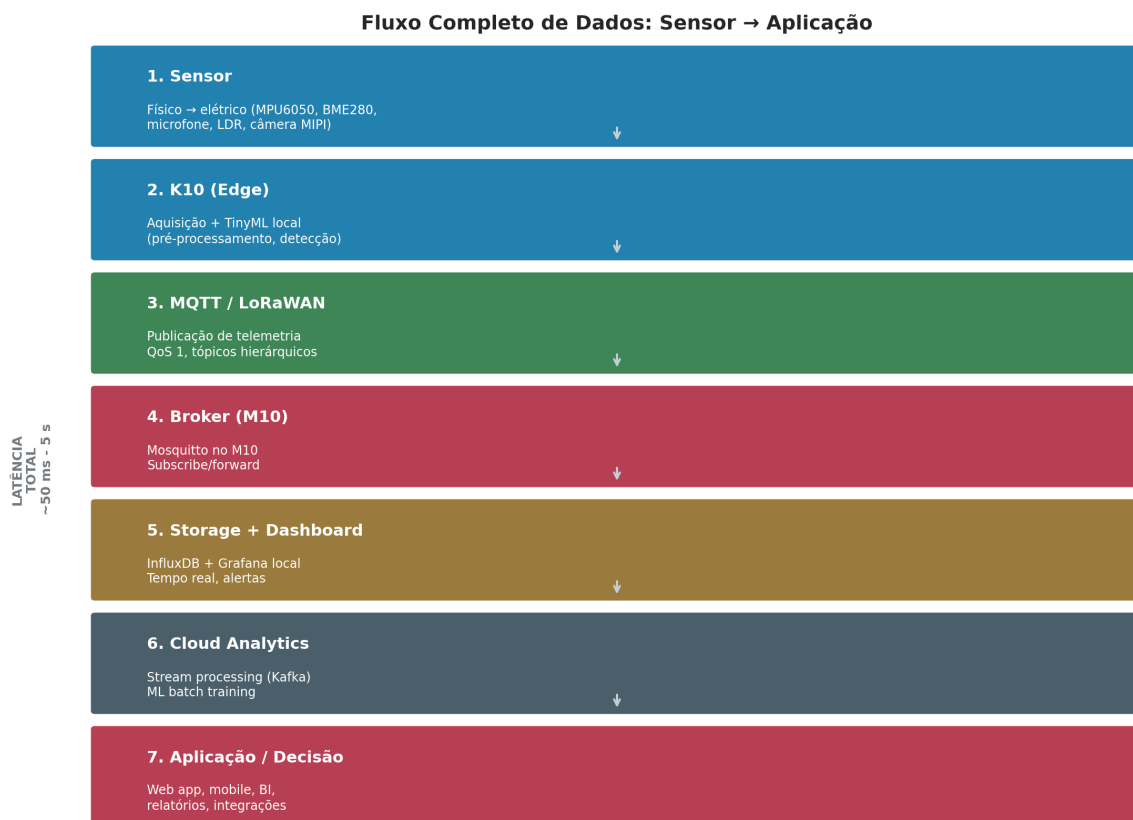


Figura 6.11 — Fluxo completo de dados em sete etapas: Sensor → K10 (Edge) → MQTT/LoRaWAN → Broker (M10) → Storage+Dashboard → Cloud Analytics → Aplicação. Fonte: elaborado pelo autor.

6.6.1 Anatomia do Fluxo

A etapa **1 (Sensor)** converte fenômeno físico em sinal elétrico (analógico ou digital). A etapa **2 (K10 Edge)** faz aquisição (via I²C, SPI, ADC, I²S), pré-processamento (filtragem, calibração) e TinyML (detecção de eventos, classificação). A etapa **3 (MQTT/LoRaWAN)** publica apenas dados significativos em tópicos hierárquicos (ex.: *fabril/linha1/maquina3/vibracao*). A etapa **4 (Broker no M10)** recebe publicações, distribui a subscribers, persiste localmente. A etapa **5 (Storage + Dashboard)** armazena em InfluxDB (time-series), visualiza em Grafana local. A etapa **6 (Cloud Analytics)** faz stream processing, ML batch training. A etapa **7 (Aplicação)** apresenta ao usuário final via web app, mobile, BI.

■ ENGENHARIA EM FOCO

Padrão arquitetural fundamental: **filtrar no Edge, agregar no Fog, persistir na Cloud**. K10 deve transmitir apenas eventos significativos (‘máquina 3 vibrando acima do normal’), não streams contínuos (‘medição de vibração a cada 100 ms’). Isso reduz volume de dados em 100-1000x, viabilizando economicamente redes com milhares de dispositivos. Transmitir tudo para Cloud é anti-pattern que quebra orçamentos de cloud e esgota largura de banda.

6.7 Arquiteturas de IA

Sistemas inteligentes frequentemente combinam múltiplas arquiteturas de IA em camadas. Esta seção analisa as principais arquiteturas e onde se encaixam em sistemas UNIHAKER.

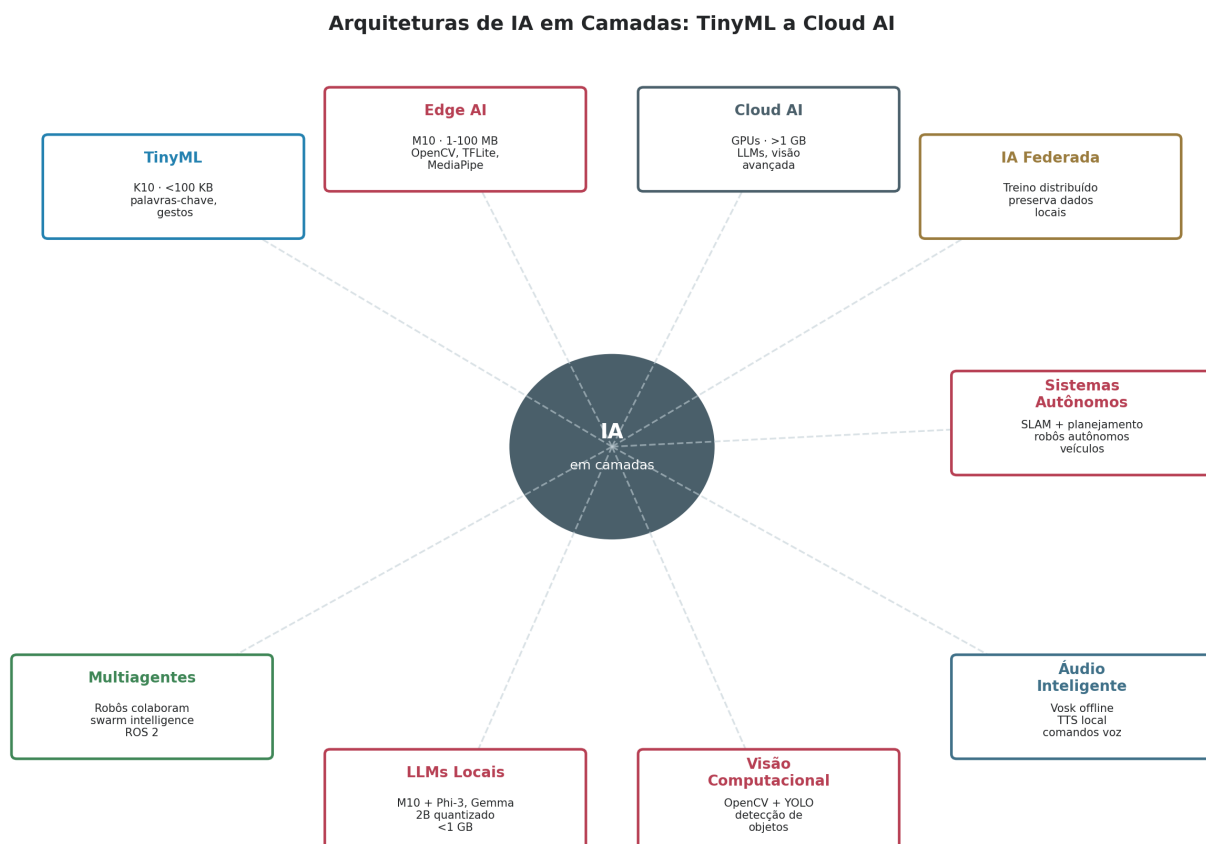


Figura 6.12 — Arquiteturas de IA em camadas, do TinyML (K10, <100 KB) ao Cloud AI (GPUs, >1 GB). Padrões emergentes (IA Federada, Multiagentes, LLMs locais) complementam o espectro. Fonte: elaborado pelo autor.

6.7.1 Camadas de IA

TinyML (K10) executa modelos <100 KB em microcontroladores: classificação de palavras-chave, detecção de gestos, anomalias em séries temporais. **Edge AI** (M10) executa modelos 1-100 MB em Linux: OpenCV, TFLite, MediaPipe, YOLO-tiny. **Cloud AI** executa modelos >1 GB em GPUs: LLMs, visão avançada, treinamento. **IA Federada** distribui treinamento entre dispositivos preservando privacidade — cada dispositivo treina localmente, compartilha apenas gradientes. **Multiagentes** coordenam múltiplos robôs ou sistemas autônomos. **LLMs Locais** (M10 + Phi-3/Gemma 2B quantizado) executam modelos de linguagem pequenos offline. **Visão Computacional** (M10 + OpenCV) detecta/segmenta objetos. **Áudio Inteligente** (M10 + Vosk) faz speech-to-text offline. **Sistemas Autônomos** combinam SLAM + planejamento para robôs e veículos.

★ APLICAÇÃO REAL

Aplicação real: sistemas de monitoramento industrial combinam as três camadas. TinyML no K10 detecta vibração anômala localmente (latência 10 ms). Edge AI no M10 classifica tipo de falha via CNN (latência 200 ms). Cloud AI treina modelo preditivo com dados históricos de toda a frota de máquinas (latência horas/dias). Cada camada tem papel irreplaceável; nenhuma sozinha é suficiente. Empresas como Siemens, GE Digital e ABB usam essa arquitetura em produtos comerciais.

6.8 Integração com Plataformas

Sistemas inteligentes raramente são construídos do zero. Integração com plataformas existentes (open source e cloud) acelera desenvolvimento e reduz risco. Esta seção analisa as principais plataformas e como o M10 (e K10, indiretamente) se integram.

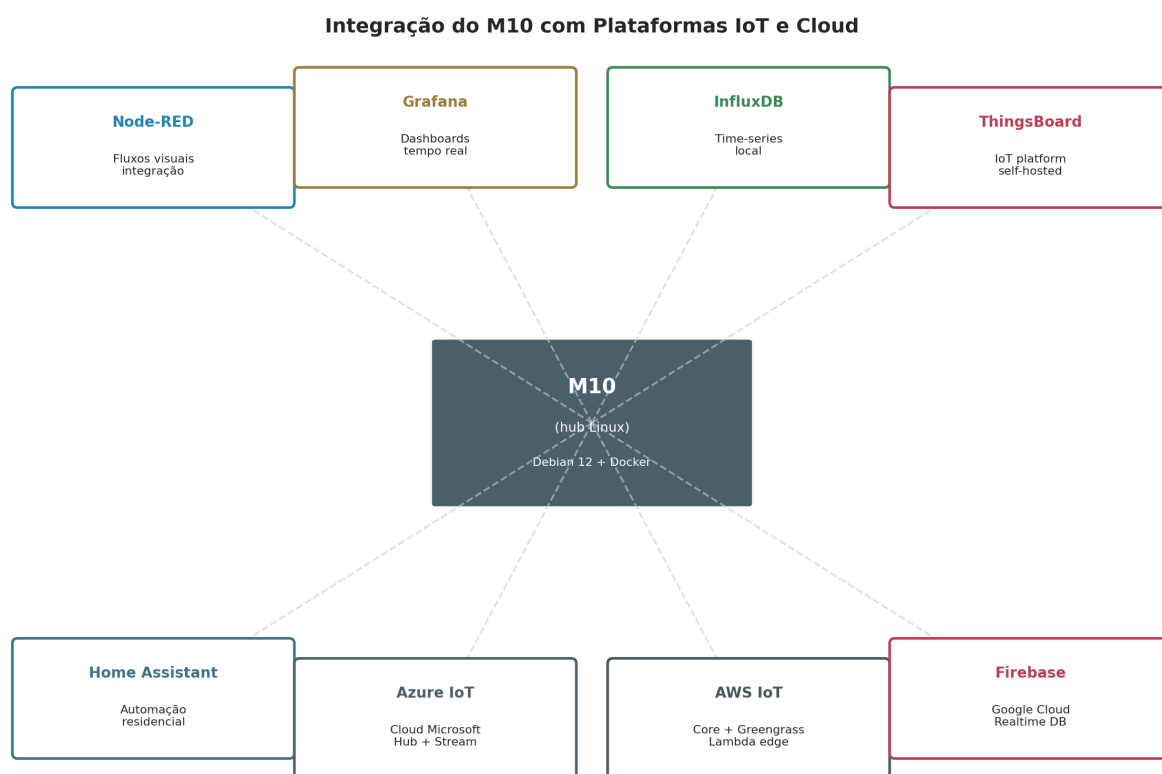


Figura 6.13 — Integração do M10 com oito plataformas IoT e Cloud. M10 atua como hub Linux central, conectando-se a Node-RED, Grafana, InfluxDB, ThingsBoard, Home Assistant, Azure IoT, AWS IoT, Firebase. Fonte: elaborado pelo autor.

6.8.1 Plataformas Open Source Self-Hosted

Node-RED é ambiente de programação visual por fluxos, instalável no M10 via apt. Permite integrar dispositivos heterogêneos (MQTT, HTTP, Modbus, OPC-UA) arrastando e conectando nós. **Grafana** é plataforma de visualização de time-series, com dashboards customizáveis, alertas, anotações. **InfluxDB** é banco de dados time-series otimizado para telemetria IoT, com compressão eficiente e query language Flux.

ThingsBoard é plataforma IoT completa (device management, dashboards, regras, alertas) self-hosted. **Home Assistant** é plataforma de automação residencial, integra centenas de dispositivos comerciais. Todos esses rodam nativamente no M10 via Docker ou apt.

6.8.2 Plataformas Cloud

Azure IoT Hub oferece device management, mensageria, stream analytics, integração com Azure ML. SDK Python compatível com M10. **AWS IoT Core** é equivalente Amazon, com Greengrass para edge computing e Lambda para serverless. **Google Cloud IoT** (descontinuado em 2023, mas alternativas como Google Cloud Pub/Sub permanecem) oferece mensageria escalável. **Firebase** é plataforma Google para aplicações real-time, com realtime database, autenticação, hosting — útil para dashboards web e mobile de sistemas IoT. **Docker** permite encapsular aplicações em containers reproduzíveis, essencial para deploy consistente. **Kubernetes** (conceitualmente, fora do escopo de M10 único) orquestra clusters de containers em escala — relevante quando sistemas crescem para dezenas de M10s.

🔗 LABORATÓRIO

Experimento sugerido: instalar Node-RED + InfluxDB + Grafana no M10 via Docker Compose. Conectar K10 (via MQTT publicando telemetria) ao Node-RED no M10. Node-RED persiste dados em InfluxDB e expõe dashboard em Grafana. Esse stack 'M10 + 3 containers + K10' é a arquitetura mínima viável de um sistema IoT profissional, replicável em qualquer projeto real.

6.9 Escalabilidade

Escalabilidade é a capacidade de um sistema de crescer em número de dispositivos, volume de dados e usuários sem degradação inaceitável de performance. Esta seção analisa como sistemas UNIHAKER escalam de uma placa a redes nacionais.

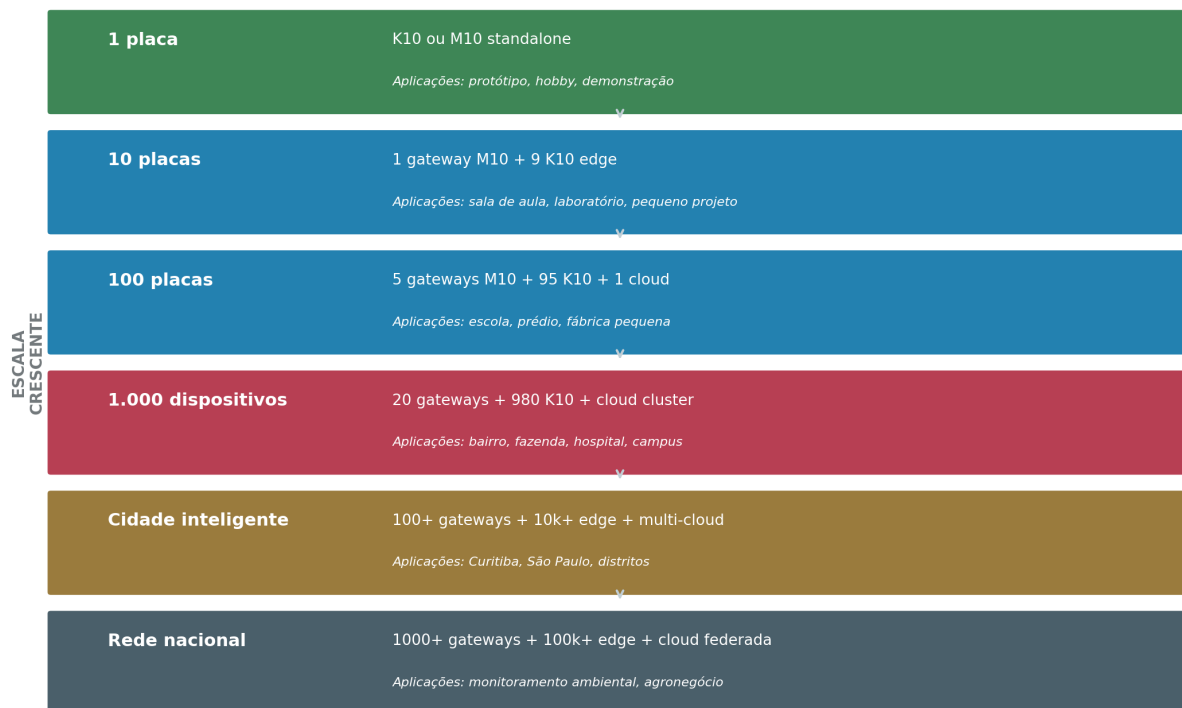
Escalabilidade: de 1 Placa a Rede Nacional

Figura 6.14 — Níveis de escalabilidade: de 1 placa (protótipo) a rede nacional (100k+ dispositivos). Cada nível introduz novos desafios arquiteturais. Fonte: elaborado pelo autor.

6.9.1 Gargalos em Cada Nível

Cada nível de escala introduz gargalos específicos. Em **1 placa**, gargalo é desenvolvimento individual. Em **10 placas**, gargalo é gerenciamento de configuração (como atualizar firmware de 10 dispositivos?). Em **100 placas**, gargalo é largura de banda de rede (100 dispositivos transmitindo simultaneamente satura Wi-Fi). Em **1.000 dispositivos**, gargalo é throughput do broker MQTT e armazenamento. Em **cidade inteligente** (10k+), gargalo é governança de dados, interoperabilidade, privacidade. Em **rede nacional** (100k+), gargalo é federalismo de dados, latência cross-region, custo de cloud.

▲ LIMITAÇÃO TÉCNICA

Anti-pattern clássico em escalabilidade IoT: ‘funciona com 10, então funciona com 10.000’. Sistemas que escalam linearmente em laboratório frequentemente colapsam em produção. Razões típicas: (i) broker MQTT único sem cluster; (ii) banco de dados sem sharding; (iii) Wi-Fi congestionado; (iv) custo de cloud exponencial; (v) ausência de cache. Solução: projetar para 10x a escala alvo desde o início, mesmo que implante em 1x inicialmente.

6.10 Segurança

Segurança em sistemas IoT é frequentemente subestimada, mas é crítica. Dispositivos IoT comprometidos podem ser usados em botnets (Mirai, 2016, comprometeu 600k dispositivos), podem vaziar dados sensíveis, podem causar danos físicos (em sistemas industriais). Esta seção analisa as camadas de segurança necessárias.

Camadas de Segurança para Sistemas IoT

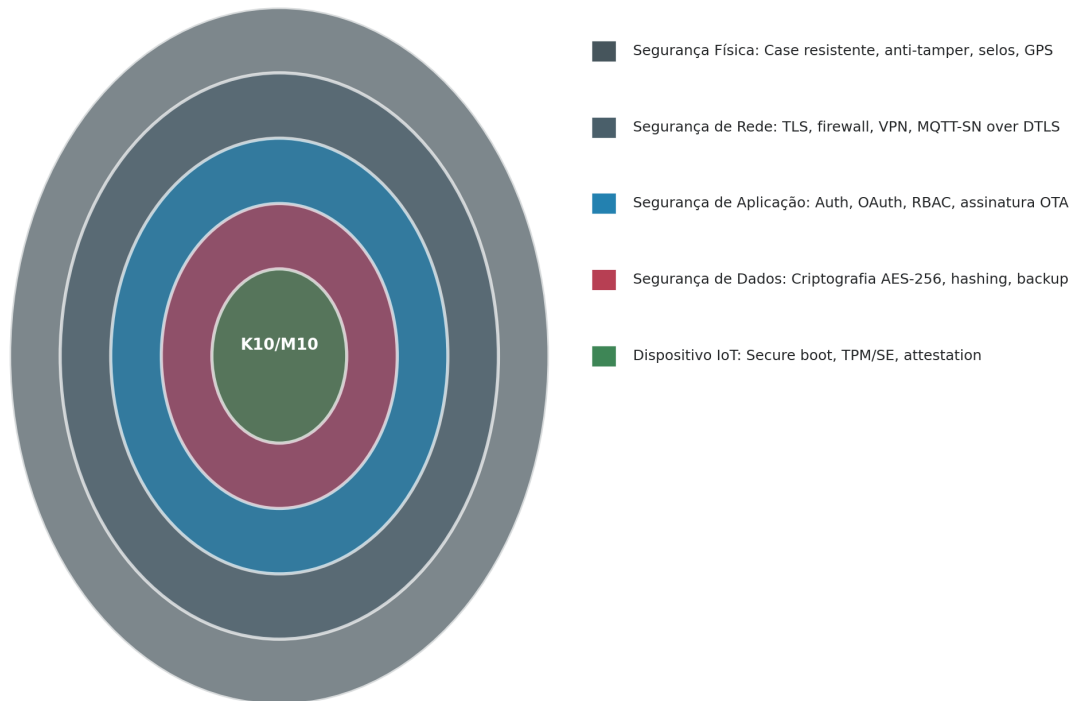


Figura 6.15 — Camadas concêntricas de segurança para sistemas IoT: física, rede, aplicação, dados, dispositivo. Cada camada deve ser protegida independentemente. Fonte: elaborado pelo autor.

6.10.1 Camadas de Segurança

Segurança física: case resistente, anti-tamper (detecção de abertura), selos de garantia, GPS em dispositivos remotos. **Segurança de rede:** TLS para todo tráfego (HTTPS, MQTT-TLS), firewall, VPN para acesso administrativo, segmentação de rede (VLANs separadas para IoT). **Segurança de aplicação:** autenticação (OAuth 2.0, JWT), autorização (RBAC), assinatura digital de firmware OTA. **Segurança de dados:** criptografia AES-256 em repouso, hashing bcrypt para senhas, backup criptografado. **Segurança de dispositivo:** secure boot (apenas firmware assinado executa), Trusted Platform Module (TPM) ou Secure Element (SE) para armazenar chaves, attestation remota.

▲ LIMITAÇÃO TÉCNICA

O K10 não possui secure boot nem TPM/SE nativos — limitação herdada do ESP32-S3. Firmware pode ser modificado por atacante com acesso físico. Mitigações: (i) case anti-tamper que desabilita dispositivo se aberto; (ii) criptografia de dados sensíveis em flash (embora chaves ainda estejam no dispositivo); (iii) attestation remota (servidor verifica hash do firmware). Para aplicações de alta segurança (saúde, financeiro), preferir M10 com secure boot configurável ou plataformas com TPM dedicado (Raspberry Pi com TPM add-on).

6.11 Engenharia das Decisões Arquiteturais

Esta seção responde às perguntas arquiteturais fundamentais que todo projeto enfrenta: quando centralizar, quando distribuir, quando usar Cloud, Edge, TinyML, Linux, IA local ou remota.

6.11.1 Centralizar ou Distribuir?

Centralizar quando: requisitos de consistência forte (transações ACID), poucos usuários, baixa latência tolerável, simplificação operacional desejada. **Distribuir** quando: latência crítica, necessidade de operação offline, volume de dados inviável de transmitir, privacidade local exigida, escalabilidade horizontal necessária. Sistemas inteligentes reais frequentemente combinam ambos: componentes distribuídos (Edge/Fog) com coordenação central (Cloud).

6.11.2 Cloud, Edge, TinyML?

Cloud quando: modelo muito grande (>1 GB), treinamento necessário, analytics multi-tenant, consolidação regional. **Edge AI (M10)** quando: modelo 1-100 MB, latência 100-500 ms aceitável, operação offline necessária, privacidade local. **TinyML (K10)** quando: modelo <100 KB, latência <100 ms crítica, bateria necessária, custo por dispositivo minimal. **Linux (M10) vs microcontrolador (K10)**: Linux quando ecossistema de software rico é necessário; microcontrolador quando determinismo, baixo consumo e simplicidade são prioritários.

DECISÃO DO ENGENHEIRO

Cenário: Startup desenvolvendo sistema de monitoramento de máquinas industriais para 50 fábricas.

Melhor escolha: arquitetura híbrida em três camadas

Justificativa: TinyML no K10 (1 por máquina, detecção local de anomalias). Edge AI no M10 (1 por fábrica, agregação + classificação de falhas). Cloud AI (treinamento do modelo com dados de todas as 50 fábricas, ML federado opcional). Cada camada tem papel irreplaceável: K10 responde em ms, M10 agrega em segundos, Cloud aprende em horas. Nenhuma camada sozinha atende todos os requisitos.

6.12 Estudos Arquiteturais

Esta seção apresenta dez estudos arquiteturais completos, cada um com arquitetura, fluxo de dados, vantagens, limitações, escalabilidade e a caixa  Arquitetura Recomendada com o stack técnico sugerido.

6.12.1 Monitoramento de Rios Amazônicos

Sistema de monitoramento de nível, qualidade da água e precipitação em bacia hidrográfica amazônica, com nós remotos solares transmitindo via satélite ou LoRaWAN de longo alcance. Desafio: ausência de infraestrutura energética e de telecomunicações em grande parte da Amazônia.

▲ ARQUITETURA RECOMENDADA**Cenário:** Monitoramento de rios amazônicos (bacia hidrográfica)**Edge:** UNIHAKER K10 (solar + bateria LiPo 5Ah)**Gateway:** UNIHAKER M10 em torres regionais (LoRaWAN concentrator)**Cloud:** Google Cloud (BigQuery + Data Studio)**Banco:** InfluxDB local no M10 + BigQuery na cloud**Dashboard:** Grafana local + Data Studio cloud**Comunicação:** LoRaWAN Classe A (K10→M10), HTTPS (M10→Cloud)**IA:** TinyML no K10 (detectar anomalias), Cloud AI (modelos hidrológicos)**Escalabilidade:** ★★★★★ (escala para milhares de nós)

Vantagens: autonomia solar de meses, cobertura em áreas sem infraestrutura. **Limitações:** latência de transmissão satelital (minutos), custo de hardware por nó. **Escalabilidade:** cada M10 regional atende 100-500 K10s; novas regiões adicionam M10s.

6.12.2 Monitoramento de ETA

Sistema de monitoramento contínuo de ETA (Estação de Tratamento de Água) urbana: pH, turbidez, cloro, vazão, nível de reservatórios. Desafio: exigência regulatória ANVISA, necessidade de auditoria.

▲ ARQUITETURA RECOMENDADA**Cenário:** Estação de Tratamento de Água (ETA) urbana**Edge:** UNIHAKER K10 (com Modbus RTU para PLCs existentes)**Gateway:** UNIHAKER M10 (Node-RED + InfluxDB local)**Cloud:** Microsoft Azure (compliance LGPD)**Banco:** InfluxDB local (90 dias) + Azure SQL (histórico)**Dashboard:** Grafana para operadores + PowerBI para gestores**Comunicação:** Modbus RTU (K10↔PLC), MQTT-TLS (K10→M10), HTTPS (M10→Azure)**IA:** TinyML no K10 (detectar anomalias), Azure ML (manutenção preditiva)**Escalabilidade:** ★★★★★ (uma arquitetura por ETA, replicável)

6.12.3 Monitoramento de Florestas

Sistema de detecção precoce de incêndios florestais e desmatamento ilegal em unidades de conservação. Desafio: área remota, sem energia, sem celular.

▲ ARQUITETURA RECOMENDADA**Cenário:** Monitoramento de florestas (UCs - unidades de conservação)**Edge:** UNIIKER K10 (solar + LoRaWAN + sensores de fumaça/temp/gases)**Gateway:** UNIIKER M10 em torres altas (LoRaWAN gateway + 4G backup)**Cloud:** AWS IoT Core + S3 + SageMaker**Banco:** InfluxDB local (M10) + DynamoDB (AWS)**Dashboard:** Grafana para ICMBio + QuickSight para público**Comunicação:** LoRaWAN (K10→M10), MQTT-TLS sobre 4G (M10→AWS)**IA:** TinyML (K10: distinguir fumaça de neblina), Rekognition (AWS: visão satélite)**Escalabilidade:** ★★★★★ (1000+ nós por UC)

6.12.4 Smart Campus Universitário

Sistema de gestão inteligente de campus universitário: iluminação, ar condicionado, ocupação de salas, qualidade do ar, mobilidade. Desafio: integração com sistemas acadêmicos existentes.

▲ ARQUITETURA RECOMENDADA**Cenário:** Smart Campus universitário**Edge:** UNIIKER K10 em salas (ocupação, CO2, temperatura)**Gateway:** UNIIKER M10 por prédio (agrega + controla HVAC)**Cloud:** Google Cloud (Firebase + BigQuery)**Banco:** InfluxDB local + Firebase Realtime DB**Dashboard:** Grafana (operação) + Looker (gestão acadêmica)**Comunicação:** Wi-Fi (K10→M10), HTTPS (M10→Cloud)**IA:** TinyML (K10: detectar ocupação por áudio), Edge AI (M10: otimizar HVAC)**Escalabilidade:** ★★★★★ (1 M10 por prédio, ~50 K10s por prédio)

6.12.5 Fábrica Inteligente

Sistema de monitoramento e otimização de linha de produção industrial: OEE (Overall Equipment Effectiveness), manutenção preditiva, qualidade.

▲ ARQUITETURA RECOMENDADA**Cenário:** Fábrica inteligente (linha de produção)**Edge:** UNIIKER K10 por máquina (vibração, corrente, temperatura)**Gateway:** UNIIKER M10 por linha (Node-RED + Docker + OPC-UA)**Cloud:** AWS IoT Core + SageMaker (manutenção preditiva)**Banco:** InfluxDB local (M10) + Timestream (AWS)**Dashboard:** Grafana (chão de fábrica) + PowerBI (gestão)**Comunicação:** Modbus RTU (K10↔PLC), MQTT Sparkplug B (K10→M10), HTTPS (M10→AWS)**IA:** TinyML (K10: anomalias), Edge AI (M10: classificar falhas), SageMaker (predição)**Escalabilidade:** ★★★★★ (1 M10 por linha, 10-50 K10s por linha)

6.12.6 Hospital Inteligente

Sistema de monitoramento de pacientes, equipamentos médicos e ambiente hospitalar. Desafio: conformidade regulatória ANVISA e LGPD.

▲ ARQUITETURA RECOMENDADA

Cenário: Hospital inteligente

Edge: UNIIKER K10 em leitos (PPG, SpO2, temperatura paciente)

Gateway: UNIIKER M10 por ala (Node-RED + alertas)

Cloud: Azure (HIPAA + LGPD compliance)

Banco: InfluxDB local + Azure Cosmos DB

Dashboard: Grafana (enfermeiros) + PowerBI (gestão)

Comunicação: BLE 5.0 (K10↔M10), HTTPS-TLS (M10→Azure)

IA: TinyML (K10: arritmias), Edge AI (M10: alertas), Azure ML (modelos)

Escalabilidade: ★★★★★ (1 M10 por ala, K10 por leito)

6.12.7 Cidade Inteligente

Plataforma urbana integrada: qualidade do ar, trânsito, iluminação, resíduos, mobilidade. Desafio: governança multi-stakeholder (prefeitura, empresas, cidadãos).

▲ ARQUITETURA RECOMENDADA

Cenário: Cidade inteligente metropolitana

Edge: UNIIKER K10 em postes (PM2.5, CO2, ruído, contagem veicular)

Gateway: UNIIKER M10 por bairro (LoRaWAN + agregação)

Cloud: Google Cloud (BigQuery + Looker)

Banco: InfluxDB (M10) + BigQuery (cloud)

Dashboard: Grafana (prefeitura) + portal público (Looker)

Comunicação: LoRaWAN (K10→M10), HTTPS (M10→Cloud)

IA: TinyML (K10: classificar qualidade do ar), Cloud AI (modelos urbanos)

Escalabilidade: ★★★★★ (100+ M10s, 10k+ K10s)

6.12.8 Agricultura de Precisão

Sistema de agricultura de precisão em larga escala: irrigação automatizada, detecção de pragas, previsão de colheita.

▲ ARQUITETURA RECOMENDADA

Cenário: Agricultura de precisão (fazenda 1000+ hectares)

Edge: UNIIKER K10 solar (umidade solo, BME280, anemômetro)

Gateway: UNIIKER M10 regional (LoRaWAN + controle de irrigação)

Cloud: AWS (S3 + SageMaker + satellite imagery)

Banco: InfluxDB local + Amazon Timestream

Dashboard: Grafana (agrônomo) + QuickSight (gestor)

Comunicação: LoRaWAN (K10→M10), HTTPS (M10→AWS)

IA: TinyML (K10: prever irrigação), Edge AI (M10: visão praga), SageMaker (safra)

Escalabilidade: ★★★★★ (1 M10 por região, 100+ K10s por M10)

6.12.9 Robótica Educacional

Sistema de robótica educacional para rede escolar: robôs controlados por alunos, com dashboards do professor.

▲ ARQUITETURA RECOMENDADA

Cenário: Robótica educacional (rede escolar)

Edge: UNIIKER K10 em robôs (motores, sensores, BLE)

Gateway: UNIIKER M10 por escola (coordenação + dashboard)

Cloud: Google Firebase (sincronização entre escolas)

Banco: Firebase Realtime DB + SQLite local (M10)

Dashboard: Web app React (professor visualiza turmas)

Comunicação: BLE 5.0 (K10↔M10), HTTPS (M10→Firebase)

IA: TinyML (K10: comandos de voz para robô)

Escalabilidade: ★★★ (1 M10 por escola, K10 por robô)

6.12.10 Assistente IA Offline

Assistente de IA local para ambiente sem internet: comandos de voz, automação, respostas a perguntas frequentes. Desafio: privacidade total (nada sai do dispositivo).

▲ ARQUITETURA RECOMENDADA

Cenário: Assistente IA offline (privacidade total)

Edge: UNIIKER K10 (wake word via TinyML, microfone sempre ativo)

Gateway: UNIIKER M10 (Vosk STT + LLM Phi-3 local)

Cloud: (nenhuma — todo processamento é local)

Banco: SQLite local + ChromaDB (vector store)

Dashboard: Display IPS do M10 + speaker

Comunicação: BLE (K10→M10), sem cloud

IA: TinyML (K10: wake word), LLM Phi-3 2B quantizado (M10)

Escalabilidade: ★★ (uma unidade por ambiente)

6.13 Conclusão

6.13.1 Protótipo vs Sistema de Engenharia

A diferença entre um protótipo e um sistema de engenharia não está na funcionalidade — ambos ‘funcionam’. Está na capacidade de operar em escala, com segurança, manutenibilidade e resiliência a falhas. Protótipo é construído para demonstrar viabilidade técnica; sistema de engenharia é construído para operar em produção por anos, atendendo milhares de usuários, sob ataques, falhas e mudanças de requisitos. O protótipo responde ‘isto é possível?’, o sistema de engenharia responde ‘isto é viável manter em produção por 5+ anos?’. Arquitetura é o que distingue um do outro.

6.13.2 Por que arquitetura é mais importante que código?

Código pode ser reescrito. Arquitetura, uma vez implantada em produção com milhares de dispositivos, é extremamente cara de mudar. Escolher errado o protocolo de comunicação (HTTP ao invés de MQTT) implica em reflashar firmware de milhares de dispositivos. Escolher errado o banco de dados (SQL ao invés de time-series) implica em migração de terabytes. Escolher errado a segmentação edge/cloud implica em rework de toda a lógica de negócio. Código é tática; arquitetura é estratégia. Projetos falham por má arquitetura muito mais frequentemente do que por mau código.

◆ INSIGHT DO ENGENHEIRO

Arquiteturas bem projetadas reduzem custos, riscos e retrabalho de três maneiras. Primeiro, **reduzem custos**: filtragem no Edge reduz custo de cloud em 100-1000x; escolha certa de protocolo reduz custo de rede; autonomia solar reduz custo de manutenção. Segundo, **reduzem riscos**: camadas de segurança dificultam ataques; redundância edge/cloud garante operação mesmo com falha de conectividade; modularidade isola falhas. Terceiro, **reduzem retrabalho**: decisões arquiteturais documentadas sobrevivem a mudanças de equipe; abstrações bem definidas permitem substituir componentes sem rework total; escalabilidade projetada desde o início evita rewrites when o sistema cresce.

Para o leitor que chegou até aqui, a mensagem final é: as plataformas UNIHAKER K10 e M10 são ferramentas poderosas, mas seu valor real emerge apenas quando integradas em arquiteturas coerentes. A pirâmide da computação, as arquiteturas de referência, os protocolos, os fluxos de dados, as camadas de IA e segurança apresentados neste capítulo são o vocabulário para projetar essas arquiteturas. Os dez estudos de caso demonstram como esse vocabulário se traduz em soluções concretas. Capítulos subsequentes aprofundarão implementação prática em projetos específicos, mas sempre sobre a base arquitetural aqui estabelecida. O engenheiro que domina esse framework está apto a projetar, justificar e implantar sistemas inteligentes em escala profissional.

Referências Bibliográficas

As referências a seguir seguem o padrão ABNT NBR 6023:2018.

- AMAZON WEB SERVICES. **AWS IoT Core developer guide**. Seattle: AWS, 2024. Disponível em: <https://docs.aws.amazon.com/iot/>. Acesso em: 5 fev. 2025.
- BALAKRISHNAN, Hari. **A primer on distributed systems**. MIT 6.824 lecture notes. Cambridge: MIT, 2023. Disponível em: <https://pdos.csail.mit.edu/6.824/>. Acesso em: 5 fev. 2025.
- BOEHM, Barry W. **Software engineering economics**. Englewood Cliffs: Prentice-Hall, 1981. Capítulo sobre custo de correção de defeitos por fase.
- DFROBOT. **UNIHAKER K10 and M10 product documentation**. Shanghai: DFRobot, 2024. Disponível em: <https://wiki.unihiker.com>. Acesso em: 5 fev. 2025.
- DOCKER. **Docker documentation: Container platform**. San Francisco: Docker Inc., 2024. Disponível em: <https://docs.docker.com>. Acesso em: 5 fev. 2025.
- ECLIPSE FOUNDATION. **Mosquitto MQTT broker documentation**. Version 2.0. Ottawa: Eclipse, 2024. Disponível em: <https://mosquitto.org/documentation/>. Acesso em: 5 fev. 2025.

- GRAFANA LABS. **Grafana documentation: Open observability platform**. New York: Grafana Labs, 2024. Disponível em: <https://grafana.com/docs/>. Acesso em: 5 fev. 2025.
- INFLUXDATA. **InfluxDB 2.x documentation: Time-series database**. San Francisco: InfluxData, 2024. Disponível em: <https://docs.influxdata.com>. Acesso em: 5 fev. 2025.
- KUBERNETES. **Kubernetes documentation: Container orchestration**. CNCF, 2024. Disponível em: <https://kubernetes.io/docs/>. Acesso em: 5 fev. 2025.
- LINUX FOUNDATION. **Edge computing reference architecture**. San Francisco: Linux Foundation, 2023. Disponível em: <https://www.lfedge.org/>. Acesso em: 5 fev. 2025.
- MICROSOFT. **Azure IoT Hub developer documentation**. Redmond: Microsoft, 2024. Disponível em: <https://learn.microsoft.com/azure/iot-hub/>. Acesso em: 5 fev. 2025.
- MQTT.ORG. **MQTT Version 5.0 specification**. OASIS, 2024. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Acesso em: 5 fev. 2025.
- NODE-RED. **Node-RED documentation: Flow-based programming for IoT**. OpenJS Foundation, 2024. Disponível em: <https://nodered.org/docs/>. Acesso em: 5 fev. 2025.
- OPC FOUNDATION. **OPC-UA specification: Industrial interoperability**. Version 1.05. Scottsdale: OPC Foundation, 2024. Disponível em: <https://opcfoundation.org/about/opc-technologies/opc-ua/>. Acesso em: 5 fev. 2025.
- SATYANARAYANAN, Mahadev. The emergence of edge computing. **IEEE Computer**, v. 50, n. 1, p. 30-39, jan. 2017. DOI: 10.1109/MC.2017.9.
- SHI, Weisong; DUSTDAR, Schahram. The promise of edge computing. **IEEE Computer**, v. 49, n. 5, p. 78-81, maio 2016. DOI: 10.1109/MC.2016.145.
- TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Distributed systems: principles and paradigms**. 3. ed. Boston: Pearson, 2017. Capítulos sobre arquitetura distribuída e middleware.
- THINGSBOARD. **ThingsBoard IoT platform documentation**. Version 3.7. New York: ThingsBoard, 2024. Disponível em: <https://thingsboard.io/docs/>. Acesso em: 5 fev. 2025.
- WARDEN, Pete; SITUNAYAKE, Daniel. **TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers**. Sebastopol: O'Reilly Media, 2020.